

data_into_silver.sql

Location: scripts/silver/data_into_silver.sql | Target Layer: Silver Layer (Standardized, Cleaned, & Validated Zone)

```
-- =====
-- ETL Script: Load & Clean Data (Bronze -> Silver)
-- Database: cadetos_silver (Reads from cadetos_bronze)
-- Project : CadetOS Data Warehouse
-- =====
-- Purpose:
--     Transforms raw data from cadetos_bronze into cleaned, standardized,
--     and validated records in cadetos_silver entirely via SQL.
--
-- SQL Data Cleaning & Transformations Performed:
--     1. CTE Deduplication: Resolves primary key duplicates and conflicts for
--       DLI numbers and transaction IDs (attendance_id, camp_id, etc.).
--     2. Temporal Constraints: Filters out parade attendance records before joining.
--     3. Rank Timeline & Demotion Filtering: standardizes rank timelines and uses
--       a running minimum window function to automatically strip demotions.
--     4. Date Standardization: Casts VARCHAR strings into native DATE formats.
--     5. Percentile-based Grade Distribution: Uses PERCENT_RANK() over attendance
--       to conform grades to 70% A, 20% B, 7% C, 3% F with huge annual variations.
--     6. Dynamic Re-appear generation: Uses UNION ALL to append passing re-attempts
--       in the subsequent year for failed cadets.
-- =====

USE cadetos_silver;

-- Clear target tables before load
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE registration;
TRUNCATE TABLE ncc_unit;
TRUNCATE TABLE attendance;
TRUNCATE TABLE camp_details;
TRUNCATE TABLE certificate_results;
TRUNCATE TABLE rank_details;
SET FOREIGN_KEY_CHECKS = 1;

-- =====
-- 1. Load Cleaned Registration Data (Deduplicated via CTE Window Functions)
-- =====

INSERT INTO registration (
    dli_number, full_name, gender, date_of_birth, aadhaar_number, email,
    mobile_number, alternate_mobile_number, city, state, pincode,
    father_name, mother_name, college, course, college_roll_number
)
WITH ranked_reg AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY dli_number ORDER BY full_name) AS rn
    FROM cadetos_bronze.registration
)
SELECT
    TRIM(dli_number) AS dli_number,
    TRIM(full_name) AS full_name,
    CASE
```

```

        WHEN UPPER(TRIM(gender)) = 'MALE' THEN 'Male'
        WHEN UPPER(TRIM(gender)) = 'FEMALE' THEN 'Female'
        ELSE TRIM(gender)
    END AS gender,
    -- Safe date parsing
    CASE
        WHEN date_of_birth REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(date_of_birth,
'%Y-%m-%d')
        WHEN date_of_birth REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(date_of_birth,
'%d/%m/%Y')
        ELSE NULL
    END AS date_of_birth,
    TRIM(aadhaar_number) AS aadhaar_number,
    LOWER(TRIM(email)) AS email,
    -- Mobile cleaning: remove non-numeric chars and take the rightmost 10 digits
    RIGHT(REGEXP_REPLACE(mobile_number, '[^0-9]', ''), 10) AS mobile_number,
    RIGHT(REGEXP_REPLACE(alternate_mobile_number, '[^0-9]', ''), 10) AS alternate_mobile_number,
    TRIM(city) AS city,
    TRIM(state) AS state,
    TRIM(pincodes) AS pincodes,
    TRIM(father_name) AS father_name,
    TRIM(mother_name) AS mother_name,
    -- College name standardization
    CASE
        WHEN UPPER(TRIM(college)) IN ('D.T.U.', 'DTU', 'DELHI TECHNOLOGICAL UNI', 'DELHI TECHNOLOGICAL
UNIVERSITY', 'DELHI TECHNOLOGICAL UNI ') THEN 'Delhi Technological University (DTU)'
        ELSE TRIM(college)
    END AS college,
    TRIM(course) AS course,
    TRIM(college_roll_number) AS college_roll_number
FROM ranked_reg
WHERE dli_number IS NOT NULL AND rn = 1;

-- =====
-- 2. Load Cleaned NCC Unit Data (Deduplicated with status-based resolution)
-- =====
INSERT INTO ncc_unit (
    dli_number, unit, directorate, group_name, squadron, cadet_year,
    joining_date, enrollment_status
)
WITH ranked_unit AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY dli_number ORDER BY
            CASE
                WHEN UPPER(TRIM(enrollment_status)) = 'ACTIVE' THEN 1
                WHEN UPPER(TRIM(enrollment_status)) = 'ENROLLED' THEN 2
                WHEN UPPER(TRIM(enrollment_status)) = 'COMPLETED' THEN 3
                WHEN UPPER(TRIM(enrollment_status)) = 'PASSED OUT' THEN 4
                WHEN UPPER(TRIM(enrollment_status)) = 'INACTIVE' THEN 5
                ELSE 6
            END ASC, cadet_year DESC) AS rn
    FROM cadetos_bronze.ncc_unit
)
SELECT
    TRIM(dli_number) AS dli_number,
```

```

    TRIM(unit) AS unit,
    TRIM(directorate) AS directorate,
    TRIM(group_name) AS group_name,
    -- Squadron name standardization
    CASE
        WHEN UPPER(TRIM(squadron)) LIKE 'ALPHA%' THEN 'Alpha'
        WHEN UPPER(TRIM(squadron)) LIKE 'BRAVO%' THEN 'Bravo'
        WHEN UPPER(TRIM(squadron)) LIKE 'CHARLIE%' THEN 'Charlie'
        WHEN UPPER(TRIM(squadron)) LIKE 'DELTA%' THEN 'Delta'
        ELSE TRIM(squadron)
    END AS squadron,
    cadet_year,
    -- Safe date parsing
    CASE
        WHEN joining_date REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(joining_date,
'%Y-%m-%d')
        WHEN joining_date REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(joining_date,
'%d/%m/%Y')
        ELSE NULL
    END AS joining_date,
    -- Enrollment status standardization
    CASE
        WHEN UPPER(TRIM(enrollment_status)) = 'ACTIVE' THEN 'Active'
        WHEN UPPER(TRIM(enrollment_status)) = 'ENROLLED' THEN 'Enrolled'
        WHEN UPPER(TRIM(enrollment_status)) = 'COMPLETED' THEN 'Completed'
        WHEN UPPER(TRIM(enrollment_status)) = 'PASSED OUT' THEN 'Passed Out'
        WHEN UPPER(TRIM(enrollment_status)) = 'INACTIVE' THEN 'Inactive'
        ELSE TRIM(enrollment_status)
    END AS enrollment_status
FROM ranked_unit
WHERE dli_number IS NOT NULL AND rn = 1;

-- =====
-- 3. Load Cleaned Attendance Data (Excludes Parade Dates before Joining Date, Deduplicated)
-- =====

INSERT INTO attendance (
    attendance_id, dli_number, parade_date, attendance_status, late, excused
)
WITH dedup_att AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY attendance_id ORDER BY attendance_status DESC) AS rn
    FROM cadetos_bronze.attendance
)
SELECT
    TRIM(attendance_id) AS attendance_id,
    TRIM(a.dli_number) AS dli_number,
    -- Safe date parsing
    CASE
        WHEN parade_date REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(parade_date, '%Y-%m-%d')
        WHEN parade_date REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(parade_date, '%d/%m/%Y')
        ELSE NULL
    END AS parade_date,
    -- Attendance status standardization
    CASE
        WHEN UPPER(TRIM(attendance_status)) IN ('PRESENT', 'P') THEN 'Present'

```

```

        WHEN UPPER(TRIM(attendance_status)) IN ('ABSENT', 'A') THEN 'Absent'
        ELSE 'Absent'
    END AS attendance_status,
    -- Late status standardization
    CASE
        WHEN UPPER(TRIM(late_status)) IN ('YES', 'Y') THEN 'Yes'
        ELSE 'No'
    END AS late,
    -- Excused leave standardization
    CASE
        WHEN UPPER(TRIM(excused_leave)) IN ('YES', 'Y') THEN 'Yes'
        ELSE 'No'
    END AS excused
FROM dedup_att a
JOIN ncc_unit nu ON a.dli_number = nu.dli_number
WHERE a.attendance_id IS NOT NULL AND a.rn = 1
    AND CASE
        WHEN parade_date REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(parade_date, '%Y-%m-%d')
        WHEN parade_date REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(parade_date, '%d/%m/%Y')
        ELSE NULL
    END >= nu.joining_date;

-- =====
-- 4. Load Cleaned Camp Details Data (Deduplicated, Typos spelling cleaned)
-- =====
INSERT INTO camp_details (
    camp_id, dli_number, camp_type, camp_location, camp_year
)
WITH dedup_camp AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY camp_id ORDER BY camp_type DESC) AS rn
    FROM cadetos_bronze.camp_details
)
SELECT
    TRIM(camp_id) AS camp_id,
    TRIM(dli_number) AS dli_number,
    -- Camp Type standardization
    CASE
        WHEN UPPER(TRIM(camp_type)) IN ('CATC', 'CAT CAMP') THEN 'CATC'
        WHEN UPPER(TRIM(camp_type)) = 'NIC' THEN 'NIC'
        WHEN UPPER(TRIM(camp_type)) = 'TSC' THEN 'TSC'
        WHEN UPPER(TRIM(camp_type)) IN ('ARMY ATTACHMENT', 'ARMY ATT.') THEN 'Army Attachment'
        WHEN UPPER(TRIM(camp_type)) = 'ALC' THEN 'ALC'
        WHEN UPPER(TRIM(camp_type)) IN ('RDC', 'REPUBLIC DAY CAMP') THEN 'RDC'
        WHEN UPPER(TRIM(camp_type)) = 'EBSB' THEN 'EBSB'
        ELSE TRIM(camp_type)
    END AS camp_type,
    -- Camp Location standardization (correct spelling errors)
    CASE
        WHEN UPPER(TRIM(camp_location)) IN ('DELHI', 'DEHLI') THEN 'Delhi'
        WHEN UPPER(TRIM(camp_location)) IN ('RAJASTHAN', 'RAJASTAN') THEN 'Rajasthan'
        WHEN UPPER(TRIM(camp_location)) IN ('KERALA', 'KERLA') THEN 'Kerala'
        WHEN UPPER(TRIM(camp_location)) IN ('PUNJAB', 'PUNJAAB') THEN 'Punjab'
        WHEN UPPER(TRIM(camp_location)) IN ('HIMACHAL PRADESH', 'HIMACHAL PRDESH') THEN 'Himachal
Pradesh'

```

```

        WHEN UPPER(TRIM(camp_location)) IN ('UTTARAKHAND', 'UTRAKHAND') THEN 'Uttarakhand'
        WHEN UPPER(TRIM(camp_location)) IN ('ASSAM', 'ASAM') THEN 'Assam'
        ELSE TRIM(camp_location)
    END AS camp_location,
    camp_year
FROM dedup_camp
WHERE camp_id IS NOT NULL AND rn = 1;

-- =====
-- 5. Load Cleaned Rank Details Data (Deduplicated, Demotions Filtered)
-- =====
INSERT INTO rank_details (
    rank_id, dli_number, rank_name, assigned_date
)
WITH raw_ranks AS (
    SELECT
        TRIM(rank_id) AS rank_id,
        TRIM(dli_number) AS dli_number,
        CASE
            WHEN UPPER(TRIM(rank_name)) = 'CADET' THEN 'CDT'
            WHEN UPPER(TRIM(rank_name)) IN ('SERGEANT', 'SGT') THEN 'SGT'
            WHEN UPPER(TRIM(rank_name)) = 'J U O' THEN 'JUO'
            WHEN UPPER(TRIM(rank_name)) IN ('L/CPL', 'LCPL') THEN 'LCPL'
            ELSE UPPER(TRIM(rank_name))
        END AS rank_name,
        -- Safe date parsing
        CASE
            WHEN assigned_date REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN STR_TO_DATE(assigned_date,
            '%Y-%m-%d')
            WHEN assigned_date REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$' THEN STR_TO_DATE(assigned_date,
            '%d/%m/%Y')
            ELSE NULL
        END AS assigned_date,
        ROW_NUMBER() OVER (PARTITION BY rank_id ORDER BY assigned_date DESC) AS rn
    FROM cadetos_bronze.rank_details
),
aligned_ranks AS (
    -- Correct rank date if assigned before joining date
    SELECT rr.*,
        CASE
            WHEN rr.assigned_date < nu.joining_date THEN nu.joining_date
            ELSE rr.assigned_date
        END AS final_assigned_date
    FROM raw_ranks rr
    JOIN ncc_unit nu ON rr.dli_number = nu.dli_number
    WHERE rr.rn = 1
),
ranked_hierarchy AS (
    -- Assign level values to ranks: SUO (1) is highest, CDT (8) is lowest
    SELECT *,
        CASE rank_name
            WHEN 'SUO' THEN 1 WHEN 'JUO' THEN 2 WHEN 'CQMS' THEN 3 WHEN 'CSM' THEN 4
            WHEN 'SGT' THEN 5 WHEN 'CPL' THEN 6 WHEN 'LCPL' THEN 7 WHEN 'CDT' THEN 8
            ELSE 8
        END AS r_level

```

```

        FROM aligned_ranks
    ),
    demotion_check AS (
        -- Get running minimum level (i.e. highest rank achieved so far)
        SELECT *,
            MIN(r_level) OVER (PARTITION BY dli_number ORDER BY final_assigned_date ROWS BETWEEN
            UNBOUNDED PRECEDING AND 1 PRECEDING) AS highest_prev_level
        FROM ranked_hierarchy
    )
    SELECT rank_id, dli_number, rank_name, final_assigned_date AS assigned_date
    FROM demotion_check
    WHERE highest_prev_level IS NULL OR r_level < highest_prev_level;

-- =====
-- 6. Load Cleaned Certificate Results (Deduplicated, Percentile Grades, Re-attempts)
-- =====
INSERT INTO certificate_results (
    certificate_id, dli_number, certificate_type, grade, passing_year
)
WITH raw_certs AS (
    SELECT
        TRIM(certificate_id) AS certificate_id,
        TRIM(dli_number) AS dli_number,
        CASE
            WHEN UPPER(TRIM(certificate_type)) IN ('A', 'A CERT') THEN 'A'
            WHEN UPPER(TRIM(certificate_type)) IN ('B', 'B CERT') THEN 'B'
            WHEN UPPER(TRIM(certificate_type)) IN ('C', 'C CERT') THEN 'C'
            ELSE UPPER(TRIM(certificate_type))
        END AS certificate_type,
        CAST(passing_year AS SIGNED) AS raw_passing_year,
        ROW_NUMBER() OVER (PARTITION BY certificate_id ORDER BY passing_year DESC) AS rn
    FROM cadetos_bronze.certificate_results
),
    joined_years AS (
        SELECT rc.*,
            YEAR(nu.joining_date) AS join_year
        FROM raw_certs rc
        LEFT JOIN ncc_unit nu ON rc.dli_number = nu.dli_number
        WHERE rc.rn = 1
    ),
    adjusted_b AS (
        SELECT *,
            CASE
                WHEN certificate_type = 'B' AND raw_passing_year <= join_year THEN join_year + 1
                ELSE raw_passing_year
            END AS b_year
        FROM joined_years
    ),
    max_b_passed AS (
        SELECT dli_number, MAX(b_year) AS max_b_yr
        FROM adjusted_b
        WHERE certificate_type = 'B'
        GROUP BY dli_number
    ),
    adjusted_timeline AS (

```

```

SELECT
    ab.certificate_id, ab.dli_number, ab.certificate_type, ab.join_year,
    CASE
        WHEN ab.certificate_type = 'A' THEN ab.join_year
        WHEN ab.certificate_type = 'B' THEN ab.b_year
        WHEN ab.certificate_type = 'C' THEN
            CASE
                WHEN ab.raw_passing_year <= COALESCE(mb.max_b_yr, ab.join_year + 1)
                    THEN COALESCE(mb.max_b_yr, ab.join_year + 1) + 1
                ELSE ab.raw_passing_year
            END
        ELSE ab.raw_passing_year
    END AS passing_year
FROM adjusted_b ab
LEFT JOIN max_b_passed mb ON ab.dli_number = mb.dli_number
),
cadet_attendance_rates AS (
    SELECT
        dli_number,
        YEAR(parade_date) AS parade_year,
        SUM(CASE WHEN attendance_status = 'Present' THEN 1 ELSE 0 END) / COUNT(*) AS att_pct
    FROM attendance
    GROUP BY dli_number, YEAR(parade_date)
),
ranked_attempts AS (
    SELECT
        at.*,
        COALESCE(car.att_pct, CAST(REGEXP_REPLACE(at.dli_number, '^[0-9]', '') AS UNSIGNED) / 1000000.0)
AS score,
        PERCENT_RANK() OVER (PARTITION BY at.passing_year ORDER BY COALESCE(car.att_pct,
CAST(REGEXP_REPLACE(at.dli_number, '^[0-9]', '') AS UNSIGNED) / 1000000.0) DESC) AS pct_rank
    FROM adjusted_timeline at
    LEFT JOIN cadet_attendance_rates car ON at.dli_number = car.dli_number AND at.passing_year =
car.parade_year
),
graded_first_attempts AS (
    SELECT
        certificate_id, dli_number, certificate_type, passing_year,
        CASE passing_year
            WHEN 2023 THEN
                CASE
                    WHEN pct_rank <= 0.80 THEN 'A'
                    WHEN pct_rank <= 0.93 THEN 'B'
                    WHEN pct_rank <= 0.98 THEN 'C'
                    ELSE 'F'
                END
            WHEN 2024 THEN
                CASE
                    WHEN pct_rank <= 0.50 THEN 'A'
                    WHEN pct_rank <= 0.85 THEN 'B'
                    WHEN pct_rank <= 0.96 THEN 'C'
                    ELSE 'F'
                END
            WHEN 2025 THEN
                CASE

```

```

        WHEN pct_rank <= 0.85 THEN 'A'
        WHEN pct_rank <= 0.94 THEN 'B'
        WHEN pct_rank <= 0.98 THEN 'C'
        ELSE 'F'
    END
    WHEN 2026 THEN
        CASE
            WHEN pct_rank <= 0.65 THEN 'A'
            WHEN pct_rank <= 0.88 THEN 'B'
            WHEN pct_rank <= 0.97 THEN 'C'
            ELSE 'F'
        END
    ELSE
        CASE
            WHEN pct_rank <= 0.70 THEN 'A'
            WHEN pct_rank <= 0.90 THEN 'B'
            WHEN pct_rank <= 0.97 THEN 'C'
            ELSE 'F'
        END
    END AS grade
FROM ranked_attempts
),
all_attempts AS (
    SELECT certificate_id, dli_number, certificate_type, grade, passing_year
    FROM graded_first_attempts
    UNION ALL
    SELECT
        CONCAT(certificate_id, '-R') AS certificate_id,
        dli_number,
        certificate_type,
        CASE MOD(CAST(REGEXP_REPLACE(dli_number, '^[0-9]', '' ) AS UNSIGNED), 10)
            WHEN 0 THEN 'C'
            WHEN 1 THEN 'B'
            WHEN 2 THEN 'B'
            ELSE 'A'
        END AS grade,
        passing_year + 1 AS passing_year
    FROM graded_first_attempts
    WHERE grade = 'F'
)
SELECT * FROM all_attempts;

-- Output conformed load metrics
SELECT 'registration' AS table_name, COUNT(*) AS record_count FROM registration
UNION ALL
SELECT 'ncc_unit', COUNT(*) FROM ncc_unit
UNION ALL
SELECT 'attendance', COUNT(*) AS record_count FROM attendance
UNION ALL
SELECT 'camp_details', COUNT(*) AS record_count FROM camp_details
UNION ALL
SELECT 'certificate_results', COUNT(*) AS record_count FROM certificate_results
UNION ALL
SELECT 'rank_details', COUNT(*) AS record_count FROM rank_details;

```